Software Re-Engineering

Fall 2026

Lecture 3

Overview of the course

- Software Change
- Software Evolution
- Software Maintenance
- Software Evolution Models and Processes
- Reengineering
- Legacy Systems
- Impact Analysis
- Refactoring
- Program Comprehension
- Software Reuse

Software Change

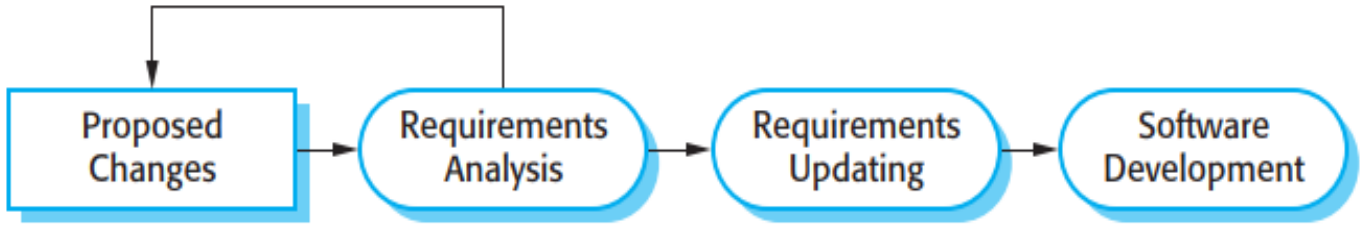
- Software development does not stop when a system is delivered but continues throughout the lifetime of the system.
- After a system has been deployed, it would require change if it is to remain useful.
- Business changes generate new requirements for the existing software.
- Parts of the software may have to be modified to correct errors that are found during operation,
- Adapt it for changes to its hardware and software platform, and to improve its performance or other quality attributes.

Software Change

- Software Change Is Inevitable
  - **New requirements emerge** as users interact with the software and their needs evolve.
  - **Errors and defects must be identified and corrected** to maintain reliable operation.
  - **Hardware and software platforms change**, requiring the system to be adapted to new environments, devices, or technologies.
  - **Performance, security, and reliability may need to be improved** as system demands increase.
  - **A major challenge for organizations is managing and implementing changes** to existing software systems while maintaining their quality, stability, and functionality.

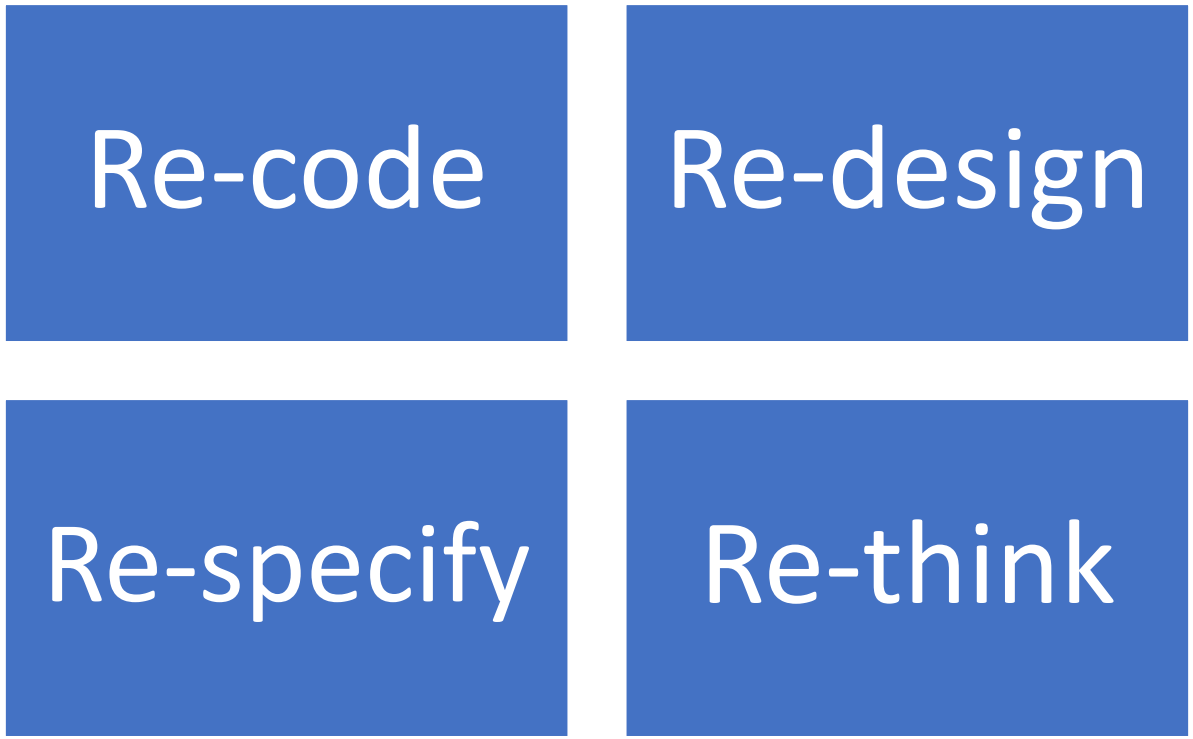
Change Implementation

- During the software **evolution process**, the proposed requirements and changes are analyzed in greater detail. This analysis may reveal additional implications or dependencies that were not identified during the initial change analysis. As a result, the proposed changes may need to be revised, and further discussions with customers or stakeholders may be required before the changes can be implemented.



Types of Change

- Based on the type and extent of changes required, software changes can be classified into four categories:



Re-code

- The characteristics or behavior of the source program are modified by **re-coding** it.
- The existing program may be **translated or transformed into another programming language** while preserving its core functionality.
- Re-coding may involve techniques such as **normalization, optimization, refactoring, and renovation**.
- The primary goal is to improve or adapt the existing code **without fundamentally changing the system's overall requirements or design**.

Re-design

- The **design characteristics of the software are modified** to accommodate significant changes in the system.
- Redesign typically focuses on improving or restructuring the system's architecture, data, or algorithms.
- Common redesign activities include:
  - **Restructuring the software architecture** to improve scalability, maintainability, or flexibility.
  - **Modifying the system's data model** to support new or changing requirements.
  - **Replacing existing procedures or algorithms** with more efficient or effective alternatives.
- The primary goal is to **improve or adapt the system's design without necessarily changing its fundamental purpose or requirements**.

Re-specify

- **Re-specification** involves changing or redefining **the requirements of the system** to reflect new or evolving needs.
- This can occur in two main ways:
  1. **Changing the form of the requirements:** Modifying how the requirements are expressed, documented, or specified while preserving their intended purpose.
  2. **Changing the scope of the requirements:** Expanding, reducing, or modifying the functionality and boundaries of the system to address new or changing user needs.

Re-think

- **Re-thinking** a system involves fundamentally reconsidering the concepts, purpose, and overall approach of an existing system.
- It involves changing the **conceptual characteristics** of the system to address a different problem domain, business need, or technological context.
- Re-thinking may require significant changes to the system's **purpose, functionality, architecture, and underlying concepts.**
  - **Example:** Moving from the development of an ordinary cellular phone to the development of a smartphone with Internet connectivity, mobile applications, multimedia capabilities, and advanced computing features.

Evolution vs. Maintenance

- The terms **software evolution** and **software maintenance** are often used interchangeably. However, there is an important semantic difference between them.
- According to Lowell Jay Arthur:
  - **Software Maintenance:** “Means to preserve software from failure or decline.”
  - **Software Evolution:** “Means a continuous change from a lesser, simpler, or worse state to a higher or better state.”
- **Maintenance:** Planned activities to preserve, correct, and improve existing software.
- **Evolution:** The continuous change and adaptation of software over time.

Software Evolution: Laws of Lehman

Continuing Change	Increasing Complexity	Self-Regulation	Conservation of Organizational Stability
Conservation of Familiarity	Continuing Growth	Declining Quality	Feedback System

Software Evolution: Laws of Lehman

1. **Continuing Change**

   A software system will gradually become less satisfactory to its users unless it is continuously adapted to meet changing requirements and needs.

2. **Increasing Complexity**

   As a system evolves, its complexity tends to increase unless deliberate efforts are made to reduce or control it.

3. **Self-Regulation**

   Software evolution is a self-regulating process, in which the development process and resulting software artifacts tend to follow predictable patterns.

4. **Conservation of Organizational Stability**

   The average level of development activity remains relatively stable over time. In other words, the amount of work required for each release tends to remain consistent.

Software Evolution: Laws of Lehman

5. **Conservation of Familiarity**

   The amount of new functionality or change introduced in each release tends to remain relatively stable or decrease over time, helping maintain user familiarity with the system.

6. **Continuing Growth**

   A software system must continually grow in functionality to satisfy changing user needs and remain useful.

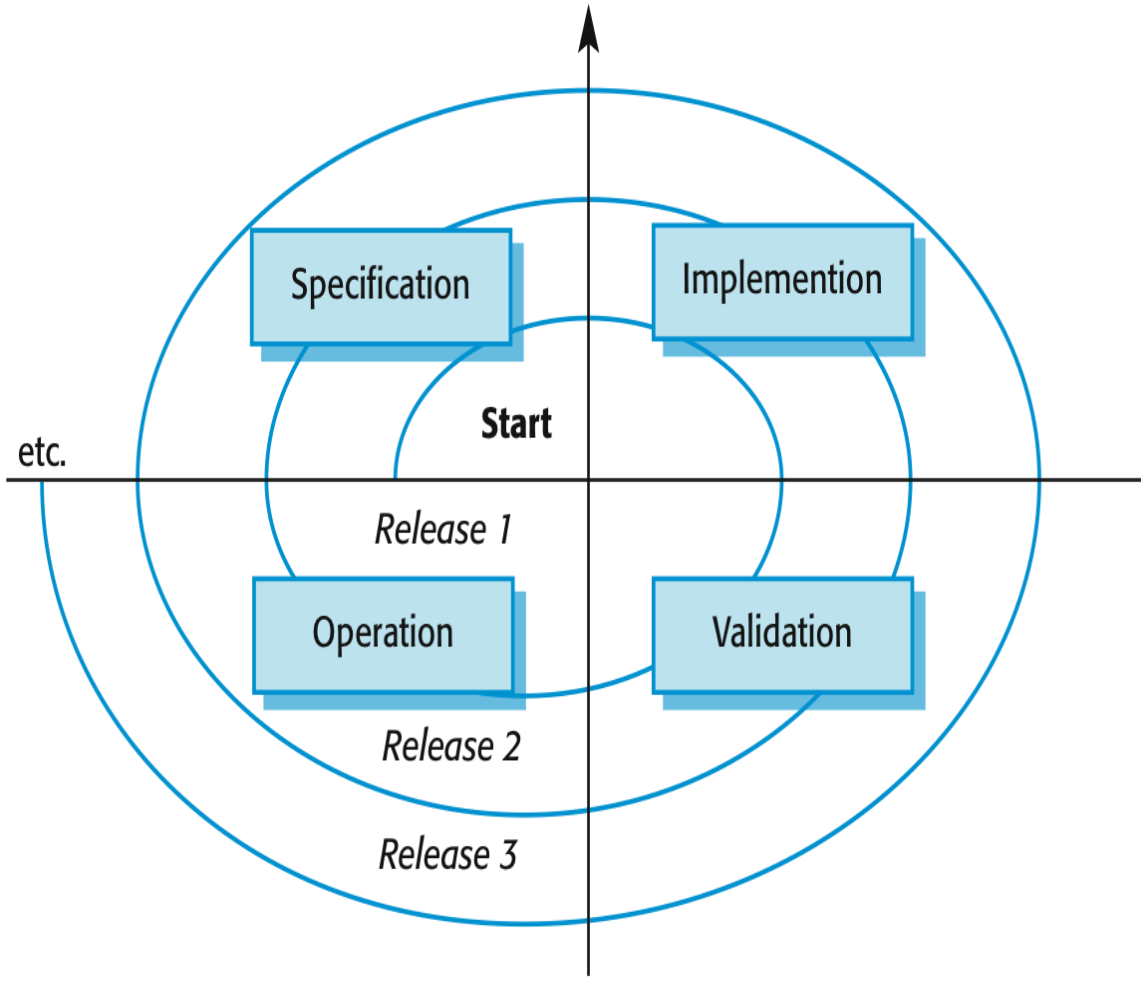
7. **Declining Quality**

   The quality of a software system tends to decline over time unless its design is continuously maintained and adapted to changing operational conditions.

8. **Feedback System**

   Successful software evolution depends on continuous feedback among developers, users, managers, and other stakeholders at different stages of the development and evolution process.

Spiral Model of Development and Evolution



Software Maintenance

- Delivered software may contain defects, as defect detection, removal, and quality assurance processes are not perfect.
- Software maintenance is therefore required to identify and fix defects and to keep the system reliable and useful over time.
- According to **E. Burton Swanson**, software maintenance can be classified into three main types:
  - **Corrective Maintenance:** Fixes errors and defects discovered after deployment.
  - **Adaptive Maintenance:** Modifies the software to work with changes in its environment, such as hardware, operating systems, or other software.
  - **Perfective Maintenance:** Improves the software by enhancing performance, usability, functionality, or maintainability.

Software Maintenance Standards

- Swanson's classification was later incorporated into ISO/IEC 14764, which introduced a fourth category: **Preventive Maintenance**.
- Some researchers and practitioners consider preventive maintenance a subset of perfective maintenance.
- **IEEE/EIA 1219** organizes the software maintenance process into seven phases:
  1. Problem Identification
  2. Analysis
  3. Design
  4. Implementation
  5. System Testing
  6. Acceptance Testing
  7. Delivery

Software Maintenance Models and Processes

- Software maintenance follows its own Software Maintenance Life Cycle (SMLC) model. Three popular SMLC models are:
  1. **Staged Model of Maintenance and Evolution**
     - Maintenance activities are performed in a sequence of well-defined stages, from identifying a change to implementing and delivering it.
  2. **Iterative Model**
     - Maintenance is carried out through repeated cycles, allowing the system to be gradually improved and refined based on feedback and new requirements.
  3. **Change Mini-Cycle Model**
     - Each change follows a small, focused cycle of analysis → design → implementation → testing, making it suitable for individual maintenance requests.

Re-engineering

- Reengineering is the process of transforming an existing software system into a new, improved form while preserving or enhancing its essential functionality.
  - It involves the examination, analysis, and restructuring of an existing software system to improve its structure, maintainability, performance, or adaptability.
  - **Chikofsky and Cross II** define reengineering as:
    - "The examination and alteration of a subject system to reconstitute it in a new form and the subsequent implementation of the new form."
- In simple terms: Reengineering means analyzing an existing system and transforming it into a better, more maintainable, and modern system.

Re-engineering

- Jacobson and Lindström describe reengineering as:

- **Reengineering = Reverse Engineering + Alteration + Forward Engineering**

- **Reverse Engineering:**
  - The process of examining an existing system to develop a more abstract and easier-to-understand representation of it. Its primary focus is on understanding and analyzing the system, rather than changing it.
- **Alteration:**
  - The process of modifying or transforming the existing system based on the required changes and improvements.
- **Forward Engineering:**
  - The traditional process of moving from high-level designs and specifications to the physical implementation of the system.

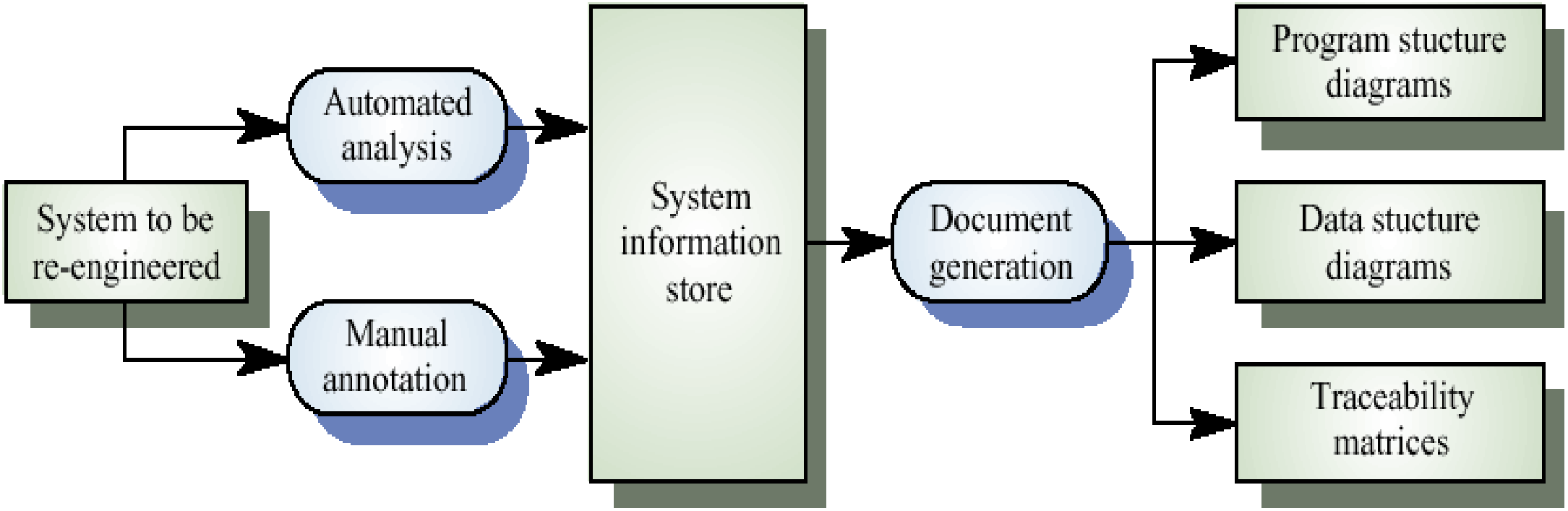
When to re-engineer

- When system changes are mostly confined to part of the system then re-engineer that part
- When hardware or software support becomes obsolete
- When tools to support re-structuring are available
- Re-engineering advantages
  - **Reduced risk:** There may be development problems, staffing problems and specification problems
  - **Reduced cost:** The cost of re-engineering is often significantly less than the costs of developing new software

Re-engineering Cost Factors

- The cost of software re-engineering depends on several factors:
- **Software Quality:** The condition, complexity, and maintainability of the existing software.
- **Tool Support:** The availability and effectiveness of tools for analysis, restructuring, and migration.
- **Data Conversion:** The amount and complexity of data that must be converted or migrated.
- **Expert Staff:** The availability and expertise of personnel required to perform the re-engineering activities.

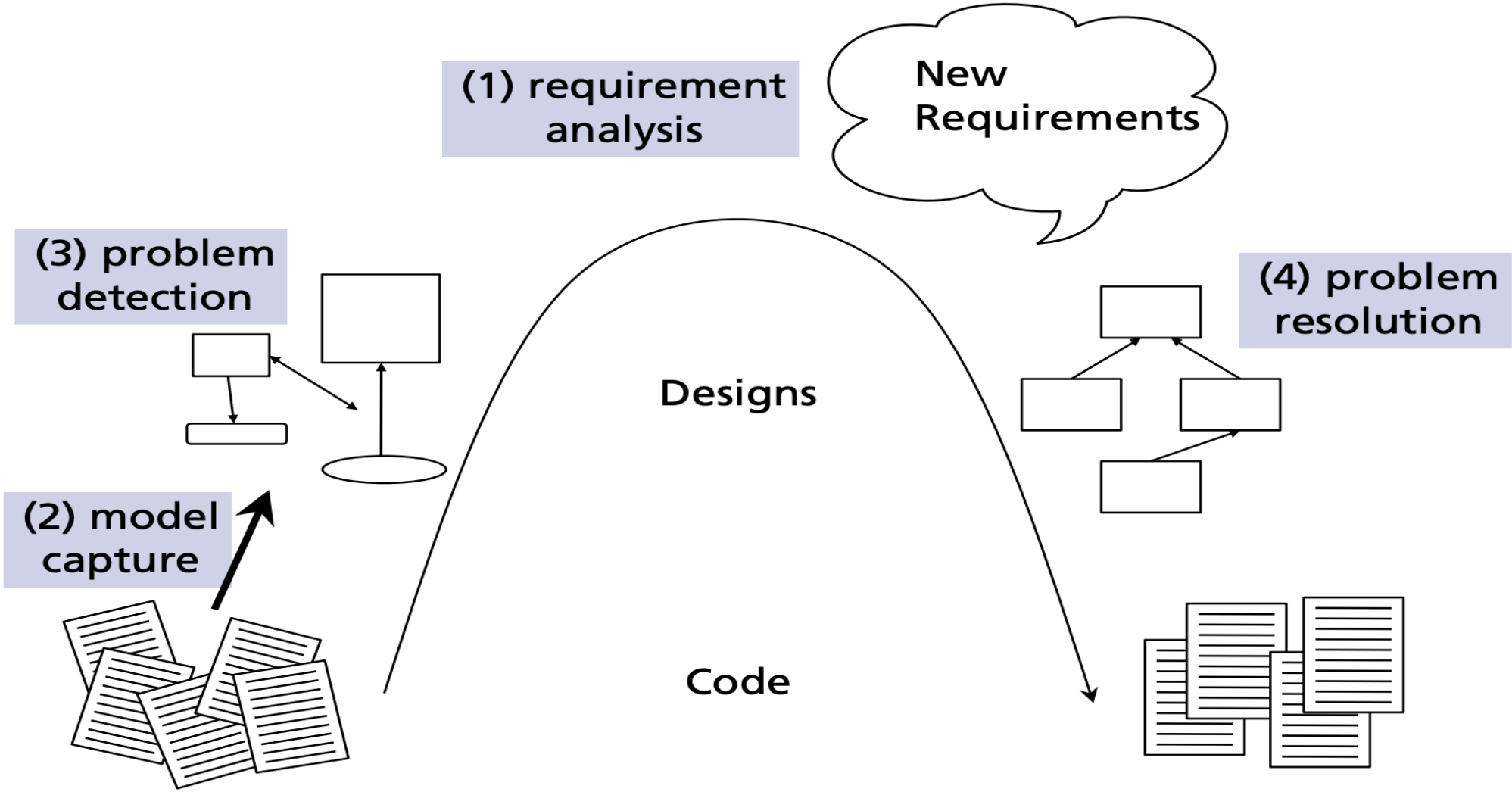
The Reverse Engineering Process



Goals of Reengineering

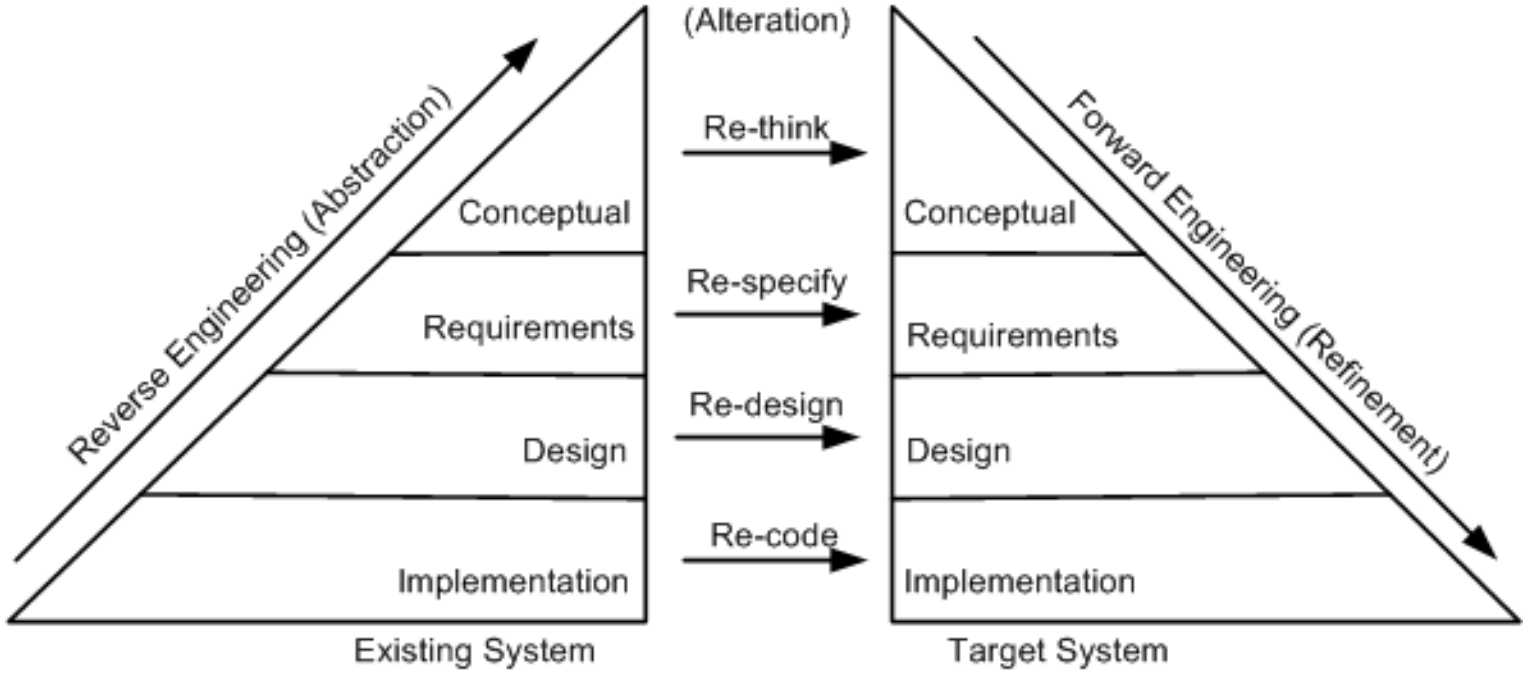
- **Unbundling**
  - Split a monolithic system into parts that can be separately marketed
- **Performance**
  - “First do it, then do it right, then do it fast”
- **Design extraction**
  - To improve maintainability, portability, etc.
- **Exploitation of New Technology**
  - I.e., new language features, standards, libraries, etc.

Reengineering Life-Cycle



A General Model for Software Reengineering

- The model proposed by Eric J. Byrne suggests that reengineering is a sequence of three activities:
- – reverse engineering, re-design, and forward engineering



Source code translation

- Involves converting the code from one language (or language version) to another e.g. FORTRAN to C
- May be necessary because of:
  - Hardware platform update
  - Staff skill shortages
  - Organizational policy changes
- Only realistic if an automatic translator is available

Data re-engineering

- Involves analyzing and reorganizing the data structures (and sometimes the data values) in a program
- May be part of the process of migrating from a file-based system to a DBMS-based system or changing from one DBMS to another
- Objective is to create a managed data environment

Legacy System

- A legacy system is an old system which is valuable for the company which often developed and owns it.
- It is the phase out stage of the software evolution model of Rajlich and Bennet described earlier.
- Bennett used the following definition:
  - “large software systems that we don’t know how to cope with but that are vital to our organization.”
- Similarly, Brodie and Stonebraker: define a legacy system as
  - “any information system that significantly resists modification and evolution to meet new and constantly changing business requirements.”

Legacy System

- There are a number of options available to manage legacy systems.
  - **Freeze:** The organization decides no further work on the legacy system should be performed.
  - **Outsource:** An organization may decide that supporting software is not core business and may outsource it to a specialist organization offering this service.
  - **Carry on maintenance:** Despite all the problems of support, the organization decides to carry on maintenance for another period.
  - **Discard and redevelop:** Throw all the software away and redevelop the application once again from scratch.
  - **Wrap:** It is a black-box modernization technique that surrounds the legacy system with a software layer that hides the unwanted complexity of the existing data, individual programs, application systems, and interfaces with the new interfaces.
  - **Migrate:** Legacy system migration basically moves an existing, operational system to a new platform, retaining the legacy system's functionality and causing minimal disruption to the existing operational business environment as possible.

Impact Analysis

- Two additional notions related to impact analysis are very common among practitioners:
  - **Ripple effect.**
  - **Change propagation.**
- Ripple effect analysis measures the impact, or how likely it is that a change to a particular module may cause problems in the rest of the program.
- Measuring ripple effect can provide knowledge about the system as a whole through its evolution:
  - how much its complexity has increased or decreased since the previous version,
  - how complex individual parts of a system are in relation to other parts of the system, and
  - to look at the effect that a new module has on the complexity of a system as a whole when it is added.
- The change propagation activity ensures that a change made in one component is propagated properly throughout the entire system.

Refactoring

- Refactoring is the process of making a change to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior.
- It is the object-oriented equivalent of restructuring.
- Refactoring, which aims to improve the internal structure of the code, achieve through the removal of duplicate code, simplification, making code easier to understand, help to find defects and adding flexibility to program faster.
- There are two aspects of the above definition:
  - It must preserve the “observable behavior” of the software system (through regression).
  - To improve the internal structure of a software system (improve maintainability).

Program Comprehension

- Program understanding or comprehension is
  - “the task of building mental models of an underlying software system at various abstraction levels, ranging from models of the code itself to ones of the underlying application domain, for software maintenance, evolution and re-engineering purposes.”
- A mental model describes a programmer’s mental representation of the program to be comprehended.
- Program comprehension deals with the cognitive processes involved in constructing a mental model of the program.
- A common element of such cognitive models is generating hypotheses and investigating whether they hold or must be rejected.
  - Hypotheses are a way to understand code in an incremental manner.
  - After some understanding of the code, the programmer forms a hypothesis and verifies it by reading code.
  - By continuously formulating new hypotheses and verifying them, the programmer understands more and more code and in increasing details.

Software Reuse

- Software reuse was introduced by Dough McIlroy in his 1968 seminal paper:
  - The development of an industry of reusable source-code software components and the industrialization of the production of application software from off-the-shelf components.
- Other significant early reuse research developments include:
  - Program families (David Parnas).
  - Domain analysis (Jim Neighbors).
- Program families are sets of programs whose common properties are so extensive that it becomes advantageous to study the common properties of these programs before analyzing individual differences.
- Whereas domain analysis is an activity of identifying objects and operations of a class of similar systems in a particular problem domain.

Cont..

- Software reuse is using existing artifacts or software knowledge during the construction of a new software system.
  - Reusable assets can be either reusable artifacts or software knowledge.
- Capers Jones identified four types of reusable artifacts:
  - data reuse, involving a standardization of data formats,
  - arhitectural reuse, which consists of standardizing a set of design and programming conventions dealing with the logical organization of software,
  - design reuse, for some common business applications, and
  - program reuse, which deals with reusing executable code.
- Software reuse of previously written code is a way to increase
  - software development productivity.
  - quality of the software.
- The cost savings during maintenance as a consequence of reuse are nearly twice the corresponding savings during development.

Cont..

- For software component to be reusable, it must exhibit the following characteristics that directly encourage its use is similar situation:
  - **Environmental independence** - The components can be reused irrespective of the environment from which they were originally captured.
  - **High cohesion** - The components that implement a single operation or set of related operations.
  - **Loose coupling** - The components that have minimal links to other components.
  - **Adaptability** - The components that are adaptable so they can be customized to fit a range of similar situation.
  - **Understandability** - The components which are easily understandable that users can quickly interpret functionality.
  - **Reliability** - The components that are error-free.
  - **Portability** - The components that are not restricted in terms of the software or hardware environment they operate in.



Questions?